JENKINS PIPELINE

Automation Testing • Practical Guide

High-contrast code edition

Java • Maven • TestNG • REST Assured • Selenium • Allure

What this guide covers

- Pipeline architecture and a reusable declarative Jenkinsfile.
- Build validation, configurable API/UI suites, reports, logs, screenshots, and cleanup.
- Jenkins setup, credentials, Allure reporting, parallel execution, troubleshooting, and interview explanations.

Important

This is a template, not a guaranteed drop-in for every repository. Update test patterns, URLs, agent setup, Maven properties, and report paths to match your framework.

Contents

01	Pipeline lifecycle
02	Complete Jenkinsfile
03	Stage-by-stage explanation
04	Jenkins and agent setup
05	JUnit and Allure reporting
06	Parallel execution
07	Credentials and environments
08	Troubleshooting
09	SDET Lead interview answers
10	Pre-run checklist

Recommended learning order

Read the lifecycle first, then follow the Jenkinsfile top to bottom. Validate one API suite and one UI suite before enabling parallel runs or release-blocking rules.

01. Pipeline lifecycle

A Jenkins pipeline automates the journey from source code to test execution and reporting. Declarative Pipeline keeps the structure consistent: pipeline, agent, options, parameters, environment, stages, and post-build actions.

Stage	Purpose	Typical output
Checkout	Fetch source code and Jenkinsfile from Git.	Commit and workspace
Validate tools	Check Java, Maven, and Git availability.	Version information
Compile	Find dependency and compilation errors early.	Compiled test classes
API tests	Execute REST Assured or other API tests.	JUnit XML and logs
UI tests	Execute Selenium browser tests.	JUnit XML and screenshots
Post actions	Publish results and retain diagnostic evidence.	Status and artifacts

Key distinction

A test failure and an infrastructure failure are not necessarily the same. Preserve evidence, report an accurate status, and make the root cause diagnosable.

Execution flow

Git checkout → tool validation → compile → API/UI execution → JUnit/Allure reporting → archive evidence → final build status.

02. Complete Jenkinsfile

Assumptions: Linux agent, Java and Maven installed, Maven Surefire reports, and test classes named *ApiTest and *UiTest. API and UI stages run sequentially in this main version for simpler output handling.

Before running

Replace the example URL. Verify the class naming patterns and make sure your framework reads the Maven properties being passed. Never hardcode passwords or tokens.

Save this as **Jenkinsfile** in your repository root:

```
pipeline {
    agent any

    options {
        timestamps()
        timeout(time: 60, unit: 'MINUTES')
        buildDiscarder(logRotator(
            numToKeepStr: '20',
            artifactNumToKeepStr: '10'
        ))
        disableConcurrentBuilds()
        skipDefaultCheckout(true)
    }

    parameters {
        choice(name: 'TEST_SUITE',
               choices: ['ALL', 'API', 'UI'],
               description: 'Suite to run')
        choice(name: 'TARGET_ENV',
               choices: ['QA', 'STAGING', 'UAT'],
               description: 'Target environment')
        choice(name: 'BROWSER',
               choices: ['chrome', 'firefox', 'edge'],
               description: 'Browser for UI tests')
        booleanParam(name: 'HEADLESS', defaultValue: true,
                     description: 'Run browser headlessly')
        string(name: 'TEST_TAG', defaultValue: '',
               description: 'Optional framework tag/group')
    }

    environment {
        BASE_URL = 'https://qa.example.com'
        MAVEN_OPTS = '-Xmx2048m'
        CI = 'true'
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
                script {
                    echo "Branch: ${env.BRANCH_NAME ?: 'manual-build'}"
                    echo "Commit: ${env.GIT_COMMIT ?: 'unknown'}"
                    echo "Suite: ${params.TEST_SUITE}"
                    echo "Environment: ${params.TARGET_ENV}"
                }
            }
        }

        stage('Validate Tools') {
            steps {
                sh '''
                    set -eu
                    java -version
                    mvn -version
                    git --version
                '''
            }
        }

        stage('Compile & Validate') {
            steps {
                sh '''
```

```
                set -eu
                mvn -B -ntp -DskipTests clean test-compile
            '''
        }
    }

    stage('API Automation') {
        when {
            expression { params.TEST_SUITE in ['ALL', 'API'] }
        }
        steps {
            catchError(buildResult: 'UNSTABLE',
                       stageResult: 'FAILURE') {
                sh '''
                    set -eu
                    mvn -B -ntp test \
                      -Dtest='*ApiTest' \
                      -DbaseUrl="${BASE_URL}" \
                      -DtargetEnv="${TARGET_ENV}" \
                      -DtestTag="${TEST_TAG}"
                '''
            }
        }
        post {
            always { echo 'API test stage completed.' }
        }
    }

    stage('UI Automation') {
        when {
            expression { params.TEST_SUITE in ['ALL', 'UI'] }
        }
        steps {
            catchError(buildResult: 'UNSTABLE',
                       stageResult: 'FAILURE') {
                sh '''
                    set -eu
                    mvn -B -ntp test \
                      -Dtest='*UiTest' \
                      -Dbrowser="${BROWSER}" \
                      -Dheadless="${HEADLESS}" \
                      -DbaseUrl="${BASE_URL}" \
                      -DtargetEnv="${TARGET_ENV}" \
                      -DtestTag="${TEST_TAG}"
                '''
            }
        }
        post {
            always { echo 'UI test stage completed.' }
        }
    }
}

post {
    always {
        junit(
            testResults: '**/target/surefire-reports/*.xml',
            allowEmptyResults: true,
            skipPublishingChecks: true
        )
        archiveArtifacts(
            artifacts: '**/target/surefire-reports/**,**/target/screenshots/**,**/target/logs/**,**/target/allure-resul
            allowEmptyArchive: true,
            fingerprint: true
        )
        echo "Final pipeline result: ${currentBuild.currentResult}"
    }
    success { echo 'Automation pipeline completed successfully.' }
    unstable { echo 'UNSTABLE: inspect failed tests and reports.' }
    failure { echo 'FAILED: inspect console output and evidence.' }
    aborted { echo 'Pipeline was aborted or interrupted.' }
    cleanup { deleteDir() }
}
}
```

Status policy

catchError lets later stages and post actions run after a suite failure, while marking the build UNSTABLE. A compile failure happens before those blocks and normally fails the pipeline. Choose the release-gate policy deliberately.

03. Stage-by-stage explanation

pipeline and agent

pipeline { } declares a Declarative Pipeline. **agent any** lets Jenkins select any eligible agent. In a controlled setup, prefer a label such as `agent { label 'linux-maven-chrome' }` to guarantee the needed tools and browser are present.

Options

- **timestamps()** adds timestamps to console output.
- **timeout** prevents a hung job from running indefinitely; tune it to suite duration.
- **buildDiscarder** limits stored builds and artifacts to manage disk usage.
- **disableConcurrentBuilds()** prevents overlapping builds of this job.
- **skipDefaultCheckout(true)** disables implicit checkout so the explicit stage controls it.

Parameters and environment

Parameters allow selection of suite, environment, browser, headless mode, and an optional tag. The test framework must read the passed system properties for them to affect runtime behaviour. Keep approved environment URLs controlled.

Checkout and tool validation

`checkout scm` checks out the SCM revision associated with the job. Java/Maven/Git version checks quickly identify a misconfigured agent.

Compile stage

`mvn -B -ntp -DskipTests clean test-compile` cleans old build output and compiles code/tests without executing tests.

API/UI test selection

The `-Dtest` patterns are examples for Maven Surefire. If your project uses TestNG XML, Cucumber tags, Maven profiles, or separate modules, replace them with the actual selection mechanism.

Post and cleanup

The post always block attempts report publication and archiving regardless of final status. The cleanup block deletes workspace files after those actions; archive anything needed for debugging first.

04. Jenkins and agent setup

Item	Configure	Purpose
Pipeline job	Pipeline from SCM or Multibranch Pipeline; correct Jenkinsfile path.	Loads pipeline code.
Git	Git plugin and repository credentials if private.	Gets source code.
Java/Maven	Compatible JDK and Maven installed/configured on agent.	Builds and runs tests.
JUnit	JUnit plugin.	Shows test counts/failures.
Credentials	Credentials and Credentials Binding plugins if secrets are needed.	Protects secrets.
Browser/Grid	Browser/driver installed or Selenium Grid configured.	Starts UI sessions.
Allure	Allure Jenkins plugin and tool configuration if used.	Publishes rich reports.

Linux vs Windows agents

This Jenkinsfile uses sh and expects a Unix-like agent. On Windows, use bat and Windows-compatible paths, or run on a prepared Linux agent. Java/Maven installation does not guarantee browsers or drivers are present.

Repository assumptions

- A valid pom.xml exists at the expected workspace root.
- Surefire writes XML under target/surefire-reports.
- Tests read properties such as baseUrl, browser, headless, targetEnv, and testTag.
- UI tests can access the configured browser or Selenium Grid.

05. JUnit and Allure reporting

JUnit XML lets Jenkins display test counts and failures. Allure provides a richer report with steps, attachments, categories, and history when the adapter and plugin are configured.

Publish JUnit

```
post {
    always {
        junit(
            testResults: '**/target/surefire-reports/*.xml',
            allowEmptyResults: true,
            skipPublishingChecks: true
        )
    }
}
```

Ensure the glob matches your actual output. Multi-module Maven projects may write reports under each module's target directory.

Publish Allure

```
stage('Publish Allure Report') {
    steps {
        allure(
            includeProperties: false,
            jdk: '',
            results: [
                [path: 'target/allure-results']
            ]
        )
    }
}
```

Requires the Allure Jenkins plugin and a results directory created by the test adapter. Add this stage after tests and before cleanup.

Raw results vs report

Archiving target/allure-results preserves raw files. The Allure Jenkins step generates and publishes the readable report. Consider retaining both.

Screenshots and logs

Configure your framework to save screenshots and logs to predictable locations such as target/screenshots and target/logs. Jenkins can archive only files that were actually created.

06. Parallel API and UI execution

Parallel execution reduces feedback time only when suites are independent. Shared workspaces and Maven output directories can collide. Prefer separate agents/workspaces, modules, unique result paths, and isolated test data.

```
stage('Parallel Automation') {
    parallel {
        stage('API Tests') {
            when {
                expression {
                    params.TEST_SUITE in ['ALL', 'API']
                }
            }
            steps {
                sh '''
                    mvn -B -ntp test \
                      -Dtest='*ApiTest' \
                      -DbaseUrl="${BASE_URL}"
                '''
            }
        }
        stage('UI Tests') {
            when {
                expression {
                    params.TEST_SUITE in ['ALL', 'UI']
                }
            }
            steps {
                sh '''
                    mvn -B -ntp test \
                      -Dtest='*UiTest' \
                      -Dbrowser="${BROWSER}" \
                      -Dheadless="${HEADLESS}" \
                      -DbaseUrl="${BASE_URL}"
                '''
            }
        }
    }
}
```

This is a structural example. For production use, isolate workspaces/agents, collect reports from both branches, and preserve your agreed failure-handling policy.

Parallel readiness

- Test users and test data are isolated.
- The target environment can handle increased load.
- Browser sessions are isolated.
- Report output paths do not collide.
- Agents have enough CPU, memory, and network capacity.

Avoid blind retries

Track flaky tests, preserve first failures, and make retries bounded and explicit.

07. Credentials and environment configuration

Store tokens, passwords, and private keys in Jenkins Credentials. Restrict access, avoid printing secrets, and use least privilege.

Bind an API token

```
withCredentials([
    string(
        credentialsId: 'automation-api-token',
        variable: 'API_TOKEN'
    )
]) {
    sh '''
        set +x
        mvn -B -ntp test
    '''
}
```

The Java framework can read API_TOKEN from the environment. It must be coded to consume the variable; this avoids placing the token in a Maven command-line property.

Map environments safely

```
environment {
    QA_BASE_URL      = 'https://qa.example.com'
    STAGING_BASE_URL = 'https://staging.example.com'
    UAT_BASE_URL     = 'https://uat.example.com'
}

stage('Resolve Environment') {
    steps {
        script {
            if (params.TARGET_ENV == 'QA') {
                env.BASE_URL = env.QA_BASE_URL
            } else if (params.TARGET_ENV == 'STAGING') {
                env.BASE_URL = env.STAGING_BASE_URL
            } else if (params.TARGET_ENV == 'UAT') {
                env.BASE_URL = env.UAT_BASE_URL
            } else {
                error('Unsupported environment')
            }
        }
    }
}
```

Add the mapping stage before tests and remove the single hardcoded BASE_URL assignment. Protect production from accidental destructive tests.

Security checklist

No secrets in source control • mask credentials in logs • use least-privilege tokens • rotate credentials • restrict production access.

08. Troubleshooting

mvn: command not found

Maven is missing from PATH or tool configuration. Verify the agent and run mvn -version.

JAVA_HOME or compile error

Align JDK, JAVA_HOME, Maven compiler release, and project requirements.

No tests executed

The test pattern may not match, or the project uses another runner/profile. Verify Surefire configuration.

No JUnit results

Check XML output location and update the junit glob.

Browser session fails

Check browser/driver versions, headless configuration, permissions, and Selenium Grid URL.

API tests fail only in Jenkins

Check DNS/network, firewall, URL, TLS, proxy, credentials, test data, and environment health.

Allure report is empty

Check adapter dependency, results directory, and Allure path.

Build green despite failures

Review catchError and status handling. Decide whether failures mean UNSTABLE or FAILURE.

Artifacts missing

Archive before cleanup and verify paths. deleteDir() removes workspace files.

Parallel output collision

Separate workspaces, output folders, and test data; aggregate results afterwards.

09. SDET Lead interview answers

Why Declarative Pipeline?

It provides a consistent structure for agents, parameters, environment, stages, and post actions. It is readable and maintainable across teams.

Smoke on commits, regression nightly?

Use a webhook or multibranch pipeline for commit/PR smoke tests and a scheduled job for nightly regression.

How are credentials secured?

Store them in Jenkins Credentials, bind only for required steps, avoid logging them, use least privilege, and rotate them.

How do you report failures?

Publish JUnit/Allure and archive evidence in post actions that run after test failures. Ensure files exist before cleanup.

How do you run in parallel?

Use parallel stages or distributed agents, isolate workspaces and test data, avoid report collisions, and aggregate results.

How do you manage flaky tests?

Measure flakiness, remove timing assumptions, use stable selectors and explicit waits, isolate data, and keep retries bounded.

Test vs infrastructure failure?

Use clear stage boundaries and logs, check tool/report availability, monitor environment health, and define status rules.

How do you secure the pipeline?

Use least-privilege SCM credentials, protected branches, controlled agents, limited production access, and auditable permissions.

Interview-ready summary

“I use a declarative Jenkins pipeline to standardise checkout, build validation, configurable API/UI execution, and reporting. Credentials stay in Jenkins Credentials, reports and evidence are published in post actions, and parallelism is introduced only after isolating workspaces and test data. I define clear status rules so genuine product failures are not hidden.”

10. Pre-run checklist

Use this checklist before connecting the pipeline to a shared or release-blocking job.

- ■ Jenkinsfile is committed at the expected repository path.
- ■ Jenkins can check out the branch and private repository if applicable.
- ■ The agent has a compatible JDK and Maven installation.
- ■ The pom.xml is in the expected workspace directory.
- ■ Test-selection patterns match actual class names or runner configuration.
- ■ Tests read the passed environment, browser, headless, and tag properties.
- ■ QA/STAGING/UAT URLs are correct and approved.
- ■ Secrets are stored in Jenkins Credentials, not source code.
- ■ Browser binaries, drivers, or Selenium Grid are available.
- ■ JUnit XML output matches the junit step's glob.
- ■ Allure adapter and result paths are configured if needed.
- ■ Screenshots and logs are saved and archived before cleanup.
- ■ Failure policy is agreed: UNSTABLE versus FAILURE.
- ■ Parallel runs have isolated workspaces, output paths, and test data.
- ■ Timeout and build retention match suite duration and storage capacity.

Final recommendation

Start with the sequential version. Get checkout, compilation, one API suite, one UI suite, JUnit publishing, and artifact retention working first. Add Allure and notifications next. Introduce parallel agents only after confirming results and test data are isolated.

Template note: Jenkins plugins, repository layout, Maven Surefire/Failsafe configuration, and test framework conventions vary. Validate commands and report paths in a non-production job before using this as a release gate.